



UNIVERSIDAD DE BUENOS AIRES

FACULTAD DE INGENIERÍA

2.º cuatrimestre de 2026

Taller de Automatización y Control

Trabajo Práctico N.º 1: Introducción al control digital

Integrantes del grupo N.º 1		
Nombre y apellido	Padrón	Correo
Francisco Javier Moya	109899	fjmoya@fi.uba.ar
Lucas Fabrizio Arias Montes	110876	lfarias@fi.uba.ar

Índice

1. Introducción	2
1.1. Componentes utilizados	2
1.1.1. Sensor ultrasónico HC-SR04	3
1.1.2. Unidad de medición inercial MPU6050	3
1.1.3. Servomotor MG996R	4
1.1.4. Arduino Nano	4
1.1.5. Potenciómetro	5
2. Limitaciones de los sensores	6
2.1. Potenciómetro	6
2.2. Sensor de distancia ultrasónico	6
2.3. Unidad de medición inercial (IMU MPU6050)	6
2.3.1. Estimación de la resolución angular	7
2.3.2. Resultados medidos y comparación	7
3. Limitaciones de los actuadores	8
3.1. Servomotor MG996R	8
3.1.1. Mapeo entre ancho de pulso y ángulo de la barra	9
3.1.2. Tiempo estimado y medido para un desplazamiento de 30°	9
3.1.3. Efecto de una frecuencia de control de 1 Hz	10
4. Limitaciones adicionales	10
5. Apéndice	12
5.1. Filtro complementario para estimar el ángulo	12
5.2. Código utilizado para estimar el tiempo de medición del potenciómetro	12
5.3. Código utilizado para estimar el tiempo de envío de 100 <code>floats</code>	12
5.4. Código de ensayo del servo y mapeo de ancho de pulso a ángulo	13

1. Introducción

El objetivo principal de la materia será controlar un sistema de tipo *Ball and Beam* como el que se observa en la figura 1.

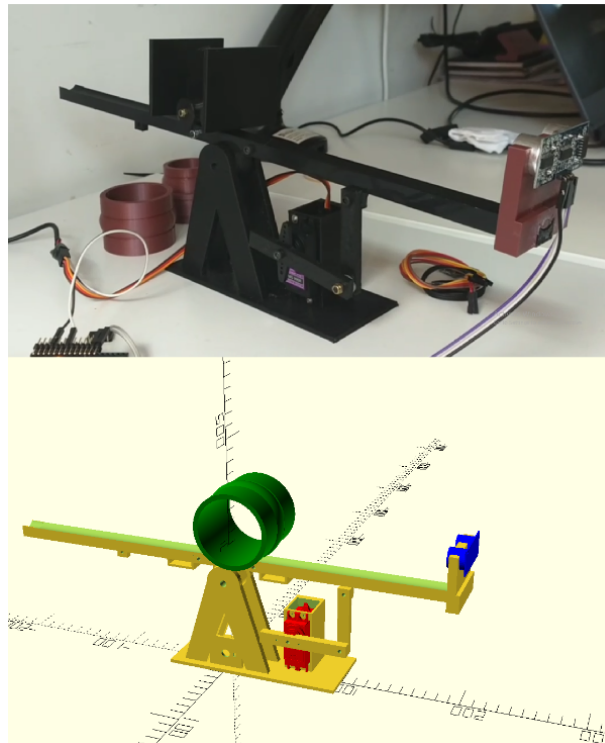


Figura 1: Sistema Ball and Beam a controlar

El sistema consta de un brazo mecánico impreso en 3D sobre el cual, en lugar de una bola, se encuentra montado un carrito que puede rodar libremente con rozamiento despreciable. Sobre el brazo están montados un sensor de distancia ultrasónico y una IMU **MPU6050**. La inclinación del brazo es controlada por un servomotor que permite que el carrito se mueva a lo largo del mismo gracias a la acción de la gravedad. Asimismo, es posible comandar un ángulo de inclinación arbitrario al brazo a través de un potenciómetro, convirtiendo la tensión medida por el mismo en un ángulo mediante un software que se ejecuta en un microcontrolador Arduino Nano, al que también se conecta toda la electrónica del sistema.

En este trabajo práctico se buscará analizar las limitaciones que presentan los sensores y actuadores que serán utilizados para controlar el sistema, así como las limitaciones adicionales que existen a la hora de enviar datos a través del puerto serie de Arduino, en particular para variables de tipo `float`. Todo ello se realiza con el fin de conocer las limitaciones que presenta el hardware con el que se va a trabajar a lo largo de la cursada y utilizar toda la información recopilada en etapas de diseño posteriores.

1.1. Componentes utilizados

Para la implementación del sistema *Ball and Beam* se utilizó un Arduino Nano como unidad de procesamiento y control. La posición del móvil sobre la barra se mide mediante un sensor ultrasónico HC-SR04, mientras que la inclinación de la barra se estima utilizando una unidad de medición inercial MPU6050. Como actuador se emplea un servomotor MG996R Digital High Torque, encargado de modificar la inclinación de la barra. Finalmente, un potenciómetro permite establecer manualmente la referencia de posición del sistema.

A continuación se presentan los principales componentes utilizados y la función que cumple cada uno dentro del sistema.

1.1.1. Sensor ultrasónico HC-SR04

Para medir la posición del móvil sobre la barra se utiliza el sensor ultrasónico HC-SR04 (fig. 2). Este dispositivo determina la distancia a un objeto a partir del tiempo de vuelo de una señal ultrasónica: el sensor emite un pulso y mide el tiempo transcurrido hasta la recepción del eco producido por el objeto [1].

El módulo posee cuatro terminales: alimentación (VCC), tierra (GND), disparo (TRIG) y recepción del eco (ECHO). El Arduino inicia cada medición mediante el terminal TRIG y determina la distancia a partir de la duración del pulso obtenido en ECHO.



Figura 2: Sensor ultrasónico HC-SR04 utilizado para medir la posición del móvil sobre la barra.

1.1.2. Unidad de medición inercial MPU6050

Para estimar la inclinación de la barra se utiliza una unidad de medición inercial (IMU) basada en el MPU6050 (fig. 3). Este dispositivo integra un acelerómetro de tres ejes y un giróscopo de tres ejes [2].

En esta aplicación, las mediciones del acelerómetro y del giróscopo se utilizan para obtener una estimación del ángulo de inclinación de la barra. La combinación de ambas mediciones permite aprovechar la información proporcionada por los dos sensores y obtener una estimación adecuada del ángulo durante el funcionamiento del sistema.

La comunicación entre el MPU6050 y el Arduino Nano se realiza mediante el protocolo I²C.

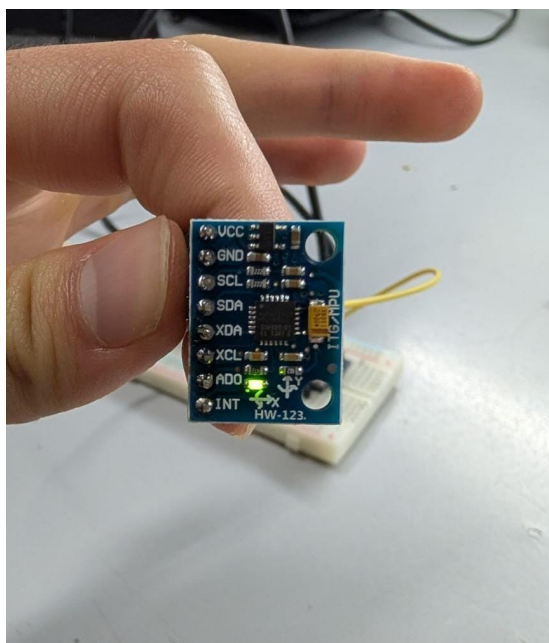


Figura 3: Unidad de medición inercial MPU6050 utilizada para estimar el ángulo de inclinación de la barra.

1.1.3. Servomotor MG996R

La inclinación de la barra se modifica mediante un servomotor MG996R *Digital High Torque*, que constituye el actuador del sistema (fig. 4). El Arduino genera la señal de comando correspondiente al ángulo deseado del servo, produciendo de esta manera el movimiento mecánico necesario para modificar la inclinación de la barra.

El análisis de la resolución angular, la señal de comando y el tiempo de respuesta del servomotor se desarrolla posteriormente en la sección correspondiente a las limitaciones de los actuadores.

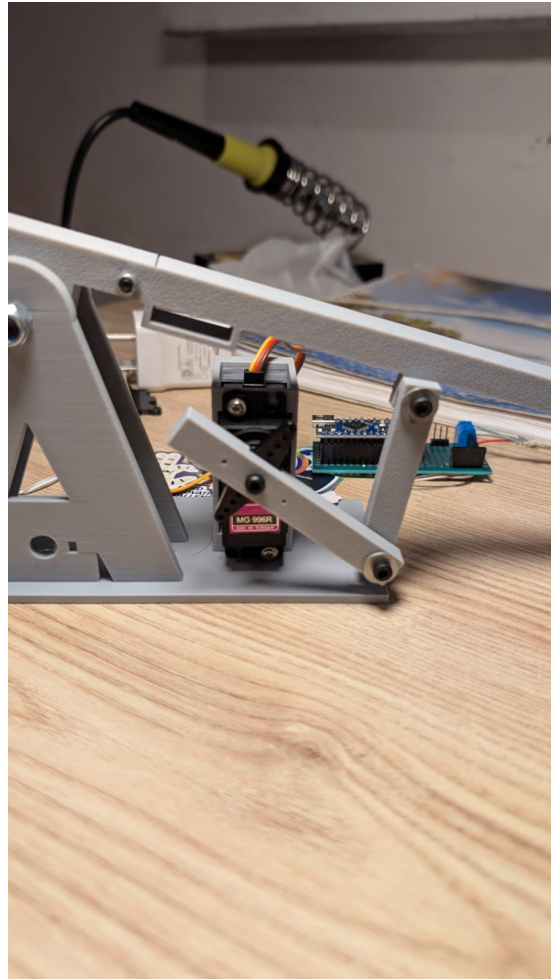


Figura 4: Servomotor MG996R utilizado como actuador del sistema.

1.1.4. Arduino Nano

El procesamiento de las mediciones y la generación de las señales de control se realizan mediante un Arduino Nano (fig. 5). El microcontrolador se encarga de adquirir las señales provenientes de los distintos sensores, procesarlas y generar la señal de comando para el servomotor.

Además, permite la comunicación con una computadora mediante el puerto serie, utilizada durante las pruebas y la adquisición de datos del sistema.

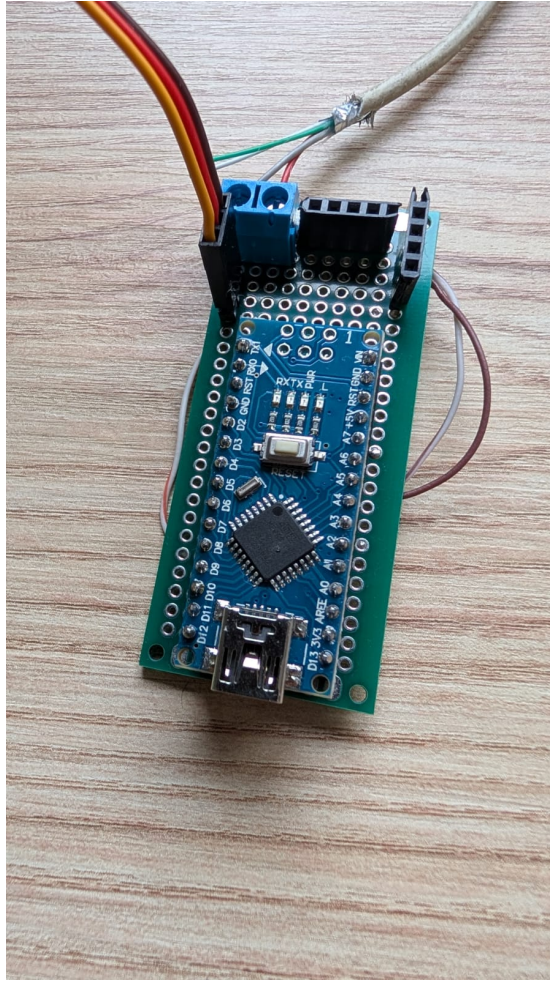


Figura 5: Arduino Nano utilizado para la adquisición y el procesamiento de datos y el control del sistema.

1.1.5. Potenciómetro

El sistema cuenta además con un potenciómetro de 3/4 de vuelta de 10 k Ω utilizado para establecer manualmente la referencia de posición del móvil (fig. 6). La tensión presente en su terminal central depende de la posición angular del mismo y es adquirida por una entrada analógica del Arduino Nano.

A partir de esta medición, el valor obtenido por el conversor analógico-digital del microcontrolador se transforma en una referencia de posición válida para el sistema.

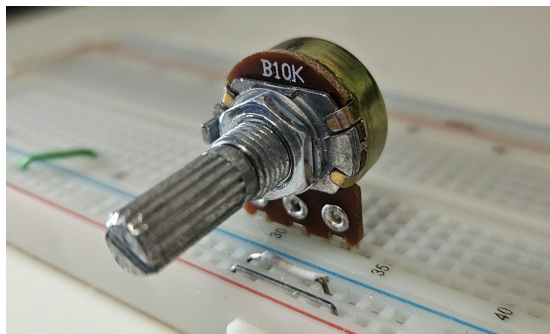


Figura 6: Potenciómetro utilizado para establecer la referencia de posición del sistema.

2. Limitaciones de los sensores

2.1. Potenci6metro

Para analizar la m6xima resoluci6n en la lectura del potenci6metro, lo primero que se debe considerar es que el conversor anal6gico-digital (ADC) del Arduino Nano posee una resoluci6n de 10 bits. Esto significa que el Arduino posee $2^{10} = 1024$ valores distintos para representar la tensi6n medida en el potenci6metro, con lo cual dicha tensi6n ser6 mapeada a un n6mero entero entre 0 y 1023. Dado que el potenci6metro se alimenta de los 5 V de referencia que provee el Arduino, la tensi6n de entrada estar6 entre 0 y 5 V, y, por ende, la m6xima resoluci6n en tensi6n alcanzable se puede calcular como:

$$\Delta V = \frac{5 \text{ V}}{1024} \approx 4,88 \text{ mV} \quad (1)$$

Para obtener un valor de 6ngulo en grados a partir del potenci6metro, se debe mapear de alguna manera la lectura obtenida del ADC a trav6s de alguna funci6n que relacione dicha lectura con un 6ngulo. Considerando que los 6ngulos a comandar al brazo estar6n en el rango $\theta \in [0^\circ, 180^\circ]$, y que el potenci6metro a utilizar es de tipo lineal, se puede utilizar la siguiente relaci6n lineal para mapear las lecturas del ADC:

$$\theta = \frac{\theta_{max} - \theta_{min}}{x_{max} - x_{min}} x = \frac{180^\circ}{1023} x \quad (2)$$

donde x representa la lectura obtenida del ADC, con $x \in [0, 1023]$.

Por ende, se deduce de la expresi6n anterior que la resoluci6n angular ser6:

$$\Delta\theta = \frac{180^\circ}{1023} \approx 0,176^\circ$$

En cuanto al tiempo de medici6n requerido, aqu6 entran en juego varios factores. En primer lugar, se debe considerar el tiempo que necesita el ADC del Arduino para transformar el valor le6do del potenci6metro a un n6mero entre 0 y 1023. Luego, hay que considerar que ese valor, adem6s, debe ser multiplicado por una constante que est6 guardada en una variable de tipo `float`, lo cual agrega un tiempo de procesamiento extra debido a que se debe manejar aritm6tica de punto flotante. Teniendo en cuenta estos factores, se estim6 experimentalmente el tiempo de medici6n utilizando el c6digo que figura en la secci6n 5.2 del ap6ndice, donde la medici6n arroj6 un valor m6nimo de 108 μs y un m6ximo de 116 μs , con lo cual:

$$\hat{t}_{med} \approx (112 \pm 4) \mu s$$

2.2. Sensor de distancia ultras6nico

A partir de la hoja de datos del sensor [1], se obtuvo que la m6xima resoluci6n en distancia es de $\Delta x = 0,3 \text{ cm}$. Considerando que la velocidad del sonido en el aire en CNPT es $v_s \approx 343 \text{ m/s}$, la m6xima resoluci6n temporal se obtiene como:

$$\Delta t = 2 \frac{\Delta x}{v_s} \approx 17,49 \mu s$$

En cuanto al tiempo de medici6n, este depender6 de la distancia a la que se encuentre el objeto, ya que la onda debe viajar hasta el carrito, rebotar en 6l y volver hasta el sensor. Considerando que la barra mide $l \approx 35 \text{ cm}$, se puede asegurar que el tiempo de medici6n ser6, como mucho, el tiempo que le tome a la onda recorrer la barra completa de ida y vuelta, por lo que:

$$t_{max} = 2 \frac{l}{v_s} \approx 2,04 \text{ ms}$$

2.3. Unidad de medici6n inercial (IMU MPU6050)

En esta secci6n se analiza qu6 cambios de 6ngulo permite distinguir la IMU y cu6nto tiempo lleva obtener una medici6n. Para este ensayo se usa solo el aceler6metro, que permite estimar la inclinaci6n de la barra a partir de la gravedad cuando est6 quieta. El gir6scopo mide velocidad angular y no se utiliza en este c6lculo.

2.3.1. Estimación de la resolución angular

El acelerómetro tiene un ADC de 16 bits por eje. Se seleccionó el rango de $\pm 2g$ para obtener la mayor sensibilidad, de 16 384 unidades digitales por g [2]. Por lo tanto, el paso mínimo de aceleración es:

$$\Delta a = \frac{g}{16\,384}.$$

En este montaje, la componente de la gravedad sobre el eje longitudinal es $a_y = g \sin(\theta)$. Cerca de la posición horizontal, $\sin(\theta) \approx \theta$, con el ángulo en radianes, de modo que $a_y \approx g\theta$. El paso angular ideal resulta:

$$\Delta\theta \approx \frac{1}{16\,384} \text{ rad} \approx 0,0035^\circ.$$

Este valor considera únicamente los pasos del ADC. En la práctica, el ruido hace que la lectura cambie incluso con la barra quieta. Con una densidad de ruido aproximada de $400 \mu g/\sqrt{\text{Hz}}$ y un ancho de banda de 44 Hz, se estima un ruido RMS (valor eficaz) de:

$$a_{\text{ruido,RMS}} \approx 400 \mu g/\sqrt{\text{Hz}} \cdot \sqrt{44 \text{ Hz}} \approx 2,65 \text{ mg}.$$

Cerca de la horizontal, esto equivale a un ruido angular de aproximadamente $0,15^\circ$. Por eso, la capacidad de distinguir pequeños cambios de ángulo queda limitada principalmente por el ruido y no por el paso del ADC.

2.3.2. Resultados medidos y comparación

Se registraron 2000 mediciones durante aproximadamente 20 segundos con la IMU quieta, luego de una espera inicial de 3 segundos. Se usó un período de muestreo de 10 ms (100 Hz), el rango de $\pm 2g$, el filtro interno a 44 Hz y el bus I²C a 400 kHz. El ángulo se calculó con las tres componentes del acelerómetro, sin filtro complementario:

$$\theta_{\text{acc}} = \text{atan2}\left(a_y, \sqrt{a_x^2 + a_z^2}\right) \frac{180^\circ}{\pi}.$$

Este cálculo supone que la aceleración medida se debe principalmente a la gravedad.

El ángulo medio fue de $-0,595404^\circ$, con una desviación estándar de $0,089478^\circ$ y una variación pico a pico de $0,681978^\circ$. La desviación estándar describe cuánto fluctúan las lecturas alrededor del promedio y permite comparar el ruido medido con el estimado. El valor medio, por sí solo, no indica el error del sensor: para conocerlo haría falta una referencia angular independiente.

Como criterio orientativo para la resolución práctica se tomó tres veces la desviación estándar:

$$\Delta\theta_{\text{práctica}} = 3\sigma \approx 0,268^\circ.$$

Para evaluar cuánto tiempo ocupa la obtención del ángulo dentro de un ciclo de control, se midieron juntas la lectura de la IMU y el cálculo del ángulo. Se utilizó `micros()`, desde antes de llamar a `getEvent()` hasta obtener el ángulo en grados. El promedio fue de $1613,55 \mu\text{s}$, con valores entre $1580 \mu\text{s}$ y $1628 \mu\text{s}$. No se incluyeron la espera inicial, el cálculo de estadísticas ni la impresión por puerto serie. Este resultado corresponde al tiempo de lectura y cálculo en el Arduino, no al retardo interno del sensor.

A una frecuencia de control de 100 Hz, cada ciclo dispone de 10 ms. El mayor tiempo observado de lectura y cálculo fue de $1,628 \text{ ms}$, aproximadamente el 16,3 % del período, dejando unos $8,37 \text{ ms}$ para las demás tareas. En las condiciones del ensayo, esta parte del programa cabe dentro del período previsto. Para confirmar que el sistema completo puede ejecutarse a 100 Hz, habrá que verificar también el tiempo de los otros sensores, el filtro, el controlador y cualquier comunicación utilizada.

La tabla 2.1 resume las estimaciones angulares y los resultados medidos. El ruido medido fue menor que el estimado, aunque del mismo orden de magnitud.

Magnitud	Estimación	Resultado experimental
Paso angular ideal del ADC	0,0035°	No se midió por separado.
Ruido angular (RMS o σ)	0,15°	0,0895°
Criterio angular de 3σ	0,45° (tres veces el ruido estimado)	0,268° (tres veces el ruido medido)
Tiempo de lectura y cálculo del ángulo	—	1,614 ms en promedio (lectura y cálculo del ángulo); rango: 1,580–1,628 ms.

Cuadro 2.1: Estimaciones angulares y mediciones de la IMU en reposo. El tiempo incluye la lectura y el cálculo del ángulo; el criterio de 3σ es orientativo.

3. Limitaciones de los actuadores

3.1. Servomotor MG996R

El actuador utilizado para modificar la inclinación de la barra es un servomotor MG996R *Digital High Torque*. Según la hoja de datos, el servomotor puede controlarse mediante una señal de pulsos periódicos, utilizando un ancho de pulso de aproximadamente 1 ms para una posición extrema, $1,5\text{ ms}$ para la posición central y 2 ms para la posición opuesta [3]. Además, el fabricante especifica un *dead band width* de $5\text{ }\mu\text{s}$, que representa una variación del ancho de pulso por debajo de la cual el cambio de posición puede no ser detectado por el servo.

Para estudiar el comportamiento del actuador en la implementación planteada, se realizó una prueba experimental variando el ancho del pulso aplicado al servomotor. Se encontró que el rango de funcionamiento utilizado por la planta se encuentra aproximadamente entre $550\text{ }\mu\text{s}$ y $1600\text{ }\mu\text{s}$. Para valores inferiores al límite mínimo, no se observaron variaciones adicionales apreciables en la inclinación de la barra, mientras que, para valores superiores al límite máximo, la barra comenzaba a inclinarse nuevamente en sentido contrario.

Tomando como referencia la posición horizontal de la barra, se obtuvo experimentalmente que un ancho de pulso de $900\text{ }\mu\text{s}$ corresponde a un ángulo de aproximadamente 0° . Los valores obtenidos para los extremos del rango utilizado fueron:

Ancho de pulso	Ángulo de la barra
$550\text{ }\mu\text{s}$	$-14,8^\circ$
$900\text{ }\mu\text{s}$	0°
$1600\text{ }\mu\text{s}$	$22,3^\circ$

Cuadro 3.1: Relación experimental entre el ancho de pulso aplicado al servomotor y el ángulo de inclinación de la barra.

La estimación del ángulo se realizó utilizando la IMU MPU6050 y un filtro complementario que combina las mediciones del acelerómetro y del giróscopo.

La hoja de datos especifica un *dead band* de $5\text{ }\mu\text{s}$ [3]. Por lo tanto, aunque sea posible modificar el ancho del pulso en pasos menores, una variación inferior a este valor puede no producir un cambio de posición observable en el eje del servomotor.

Considerando el rango de control indicado en la hoja de datos, de 1 ms a 2 ms , y tomando los 180° de recorrido aproximado indicados por el fabricante, se obtiene como estimación de la relación entre el ancho de pulso y el ángulo:

$$\frac{\Delta\theta}{\Delta PWM} = \frac{180^\circ}{1000\text{ }\mu\text{s}} = 0,18^\circ/\mu\text{s}.$$

De esta manera, una variación de $5\text{ }\mu\text{s}$, correspondiente al *dead band* especificado por el fabricante, equivale aproximadamente a:

$$\Delta\theta_{\text{dead band}} = 5\text{ }\mu\text{s} \cdot 0,18^\circ/\mu\text{s} = 0,9^\circ.$$

Por lo tanto, la resolución angular teórica asociada al *dead band* del servomotor es del orden de $0,9^\circ$. Este valor debe interpretarse como una estimación, ya que la relación entre el ancho de pulso y el ángulo real depende del servo y de la mecánica utilizada.

Además de la resolución teórica, se analizó experimentalmente cuál es la mínima variación del ancho de pulso que produce un movimiento observable en la barra. Se encontró que variaciones del ancho de pulso de aproximadamente $5 \mu s$ ya producían desplazamientos apreciables de la barra, como especificaba el fabricante. Por ejemplo, partiendo de un pulso de $900 \mu s$, tanto un incremento hasta $905 \mu s$ como una disminución hasta $895 \mu s$ generaban una variación observable en la inclinación. Por lo tanto, para la implementación planteada se considera una variación mínima observable de:

$$\Delta PWM_{\min} \approx 5 \mu s.$$

Para estimar qué variación angular representa este cambio, se pueden considerar los dos lados del punto de equilibrio por separado. Para el lado negativo se obtuvo:

$$\frac{\Delta \theta}{\Delta PWM} = \frac{14,8^\circ}{900 \mu s - 550 \mu s} = 0,0423^\circ / \mu s.$$

Por lo tanto, una variación de $5 \mu s$ representa aproximadamente:

$$\Delta \theta_{\min} \approx 5 \mu s \cdot 0,0423^\circ / \mu s \approx 0,21^\circ.$$

Para el lado positivo se obtiene:

$$\frac{\Delta \theta}{\Delta PWM} = \frac{22,3^\circ}{1600 \mu s - 900 \mu s} = 0,0319^\circ / \mu s,$$

lo que da:

$$\Delta \theta_{\min} \approx 5 \mu s \cdot 0,0319^\circ / \mu s \approx 0,16^\circ.$$

Los resultados muestran que la relación entre el ancho de pulso y el ángulo de la barra no es perfectamente lineal ni simétrica. Para la implementación propuesta, una variación de $5 \mu s$ del ancho de pulso produce aproximadamente una variación angular observable de entre $0,16^\circ$ y $0,21^\circ$.

3.1.1. Mapeo entre ancho de pulso y ángulo de la barra

Sea p el ancho del pulso alto expresado en μs . A partir de los tres puntos de la tabla 3.1, se propone una interpolación lineal por tramos que conserva la posición horizontal medida:

$$\hat{\theta}_b(p) = \begin{cases} \frac{14,8}{350}(p - 900)^\circ, & 550 \leq p \leq 900, \\ \frac{22,3}{700}(p - 900)^\circ, & 900 < p \leq 1600. \end{cases} \quad (3)$$

Por ejemplo, $895 \mu s$ corresponde a $-0,211^\circ$ y $905 \mu s$ a $0,159^\circ$. El mapeo inverso, con θ_b expresado en grados y p en μs , es:

$$p(\theta_b) = \begin{cases} 900 + \frac{350}{14,8}\theta_b, & -14,8 \leq \theta_b \leq 0, \\ 900 + \frac{700}{22,3}\theta_b, & 0 < \theta_b \leq 22,3. \end{cases} \quad (4)$$

Este modelo estima el ángulo de la barra y no se extrapola fuera del intervalo ensayado. Las pendientes son promedios de cada tramo. El ancho de pulso determina la consigna.

3.1.2. Tiempo estimado y medido para un desplazamiento de 30°

La especificación de velocidad del MG996R indica $0,17 s/60^\circ$ a $4,8 V$ y $0,14 s/60^\circ$ a $6 V$ [5]. Suponiendo velocidad constante, el tiempo de recorrido para una variación de 30° en el eje del servo se estima como:

$$\hat{t}_{30} = \frac{30^\circ}{60^\circ} t_{60} = \begin{cases} 85 ms, & V = 4,8 V, \\ 70 ms, & V = 6 V. \end{cases} \quad (5)$$

La biblioteca **Servo** mantiene un intervalo de refresco nominal de $20 ms$ [6]. Una orden puede esperar aproximadamente entre 0 y $20 ms$ hasta el siguiente pulso; sumando solo esa espera al recorrido nominal se obtienen $85\text{--}105 ms$ a $4,8 V$ y $70\text{--}90 ms$ a $6 V$, sin incluir otros retardos internos.

Cabe destacar que este cálculo constituye una aproximación, ya que la IMU utilizada para medir el desplazamiento angular se encuentra ligeramente desplazada respecto del eje de rotación de la barra. Por lo tanto, el tiempo obtenido representa una estimación del tiempo requerido para que la barra realice un desplazamiento de 30° .

Para medir la respuesta del sistema montado, se realizó este desplazamiento de -10° a 20° de la barra mediante el programa `t_respuesta_servo.ino`, registrando el ángulo estimado con la IMU y el filtro complementario cada 10 ms . Se midió el tiempo desde la orden hasta la entrada en una banda de $\pm 1^\circ$ alrededor del ángulo final, confirmando que la medición permaneciera dentro de ella durante 100 ms . En el ensayo se obtuvo:

$$t_{\text{llegada, barra}} = 391,9\text{ ms} \approx 0,39\text{ s}.$$

Este resultado corresponde a un único ensayo y debe considerarse como una estimación, ya que la medición está limitada por el período de muestreo de la IMU, de 10 ms .

3.1.3. Efecto de una frecuencia de control de 1 Hz

Se entiende por frecuencia de control la frecuencia con que se calcula y actualiza la consigna. A 1 Hz el período es $T_c = 1\text{ s}$, mientras que `Servo` continúa generando pulsos aproximadamente a 50 Hz con el último ancho solicitado. Por lo tanto, el mapeo estático, el *dead band* de $5\text{ }\mu\text{s}$ y las estimaciones de resolución angular anteriores no cambian por reducir la frecuencia de actualización. Tampoco cambia por ese motivo el tiempo nominal de recorrido de $70\text{--}85\text{ ms}$ una vez recibida la orden.

Lo que aumenta es la espera para aplicar una nueva acción. Para un cambio que ocurre entre dos actualizaciones, la espera es de 0 a casi 1 s (promedio $0,5\text{ s}$ si ocurre con fase uniforme). Sumando esa espera y el refresco del servo al recorrido nominal, la respuesta estimada para 30° del eje puede aproximarse a $1,105\text{ s}$ a $4,8\text{ V}$, o $1,090\text{ s}$ a 6 V , en el caso de mayor espera. Estos valores no son cotas garantizadas bajo carga. La consigna queda constante durante un segundo y el controlador no puede corregir perturbaciones entre actualizaciones, aunque el servo complete antes su movimiento.

Si también se adquieren los sensores a 1 Hz , sus pasos de cuantización y tiempos por lectura no cambian, pero se pierde información de la evolución entre muestras; la frecuencia de Nyquist pasa a $0,5\text{ Hz}$. Las cifras de ruido del ángulo del acelerómetro obtenidas a 100 Hz no caracterizan por sí solas el comportamiento dinámico a 1 Hz . Si se utiliza un filtro complementario, también debe ajustarse al nuevo período de muestreo y evaluarse en esa condición. Es posible mantener la IMU a 100 Hz y actualizar solo la consigna a 1 Hz , como permite el código del apéndice 5.4.

Si, en cambio, se pretende emitir los propios pulsos del servo a 1 Hz , se trata de una condición distinta del refresco utilizado aquí: no se pueden asegurar la posición, el par de mantenimiento ni los tiempos anteriores sin verificar el comportamiento del dispositivo. No se debe conservar el ciclo de trabajo al cambiar el período, porque eso alteraría el ancho del pulso y, por lo tanto, la consigna.

4. Limitaciones adicionales

Para analizar las limitaciones presentes en el envío de datos, en esta sección se estudia el tiempo requerido para transmitir `100 floats` a través del puerto serie de Arduino, a una velocidad de 115200 bps .

Dado que cada `float` ocupa `32 bits` en memoria, a la velocidad mencionada, el tiempo requerido para enviar un único `float` se calcula como:

$$t_{\text{float}} = \frac{32}{115200} \approx 277,78\text{ }\mu\text{s} \quad (6)$$

Por lo tanto, en teoría, el tiempo requerido para enviar los `100 floats` a través del puerto serie sería de aproximadamente $t_{100\text{ floats}} \approx 27,78\text{ ms}$. Para corroborar este valor, se midió dicho tiempo experimentalmente utilizando el código que se encuentra en la sección 5.3 del apéndice. De la medición se obtuvo un valor de:

$$\hat{t}_{100\text{ floats}} \approx 35\text{ ms} \quad (7)$$

Este resultado muestra una correspondencia adecuada con el valor calculado de manera teórica. De esta medición se puede obtener además que $\hat{t}_{\text{float}} \approx 350\text{ }\mu\text{s}$. No sería recomendable medir el tiempo que tarda un único `float` en ser enviado directamente, ya que eso arrojaría mediciones con un alto grado de error, dado que se estaría superando la propia resolución de la función `micros()`, que es de $4\text{ }\mu\text{s}$.

Por otro lado, cabe aclarar que la transmisión de datos en el Arduino se da de manera *asincrónica*. El método `Serial.write()` únicamente se encarga de escribir los datos en el búfer de transmisión. Luego,

dicho búfer se vacía llamando al método `Serial.flush()`, enviando así los datos a través del puerto serie. Por ello, en el código utilizado se tuvo en cuenta tanto el tiempo de escritura como lo que tarda el búfer en vaciarse para considerar que los datos fueron efectivamente enviados.

Referencias

- [1] E. J. Morgan, *HC-SR04 Ultrasonic Sensor*, 2014.
- [2] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification, Revision 3.3*, 2012.
- [3] Handson Technology, *MG996R Metal Gear Servo Motor*.
- [4] Arduino, *Arduino UNO R3 Product Reference Manual*, Rev. 3, 30/04/2024.
- [5] Tower Pro, *MG996R High Torque Metal Gear Dual Ball Bearing Servo*, https://admin.techshopbd.com/uploads/product_document/MG996R_Tower-Pro.pdf.
- [6] Arduino, *Biblioteca Servo: Servo.h, REFRESH_INTERVAL*, <https://github.com/arduino-libraries/Servo/blob/master/src/Servo.h>.
- [7] Mario Garcia, *Complementary Filter*, documentación de AHRS, <https://ahrs.readthedocs.io/en/latest/filters/complementary.html>.

5. Apéndice

5.1. Filtro complementario para estimar el ángulo

Para medir el ángulo de la barra se utilizará un filtro complementario, que combina el acelerómetro y el giróscopo de la IMU. El acelerómetro estima la inclinación a partir de la gravedad y aporta la información de bajas frecuencias (cambios lentos), pero es sensible a las vibraciones y a las aceleraciones del movimiento. Su sesgo, es decir, un desplazamiento constante en la lectura, también puede producir un error en el ángulo. El giróscopo mide la velocidad angular y permite seguir los cambios rápidos (altas frecuencias), pero al integrarla para obtener el ángulo, su sesgo acumula un error que crece con el tiempo, llamado deriva.

El filtro combina la parte de bajas frecuencias del ángulo del acelerómetro con la de altas frecuencias del ángulo obtenido del giróscopo. Así se reducen las fluctuaciones del acelerómetro y la deriva del giróscopo, obteniendo una estimación más estable [7]. Esto no elimina el sesgo del acelerómetro, por lo que sigue siendo necesario calibrar los sensores.

5.2. Código utilizado para estimar el tiempo de medición del potenciómetro

```
// inicializo variables
int sensorPin = A0;
int sensorValue = 0;
unsigned long t0 = 0;
unsigned long dif = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  t0 = micros();
  sensorValue = analogRead(sensorPin);
  float angulo = sensorValue * 180/1023;
  dif = micros() - t0;
  Serial.println( );
  Serial.print( Tiempo de medicion total: );
  Serial.println(dif);
  delayMicroseconds(20000 - micros());
}
```

5.3. Código utilizado para estimar el tiempo de envío de 100 floats

```
void sendFloat(float value) {
  const byte *bytes = reinterpret_cast<const byte *>(&value);
  Serial.write(bytes, sizeof(value));
}

void setup() {
  Serial.begin(115200);
}

void loop() {
  static unsigned long t0 = 0;
  static unsigned long t1 = 0;
  t0 = millis();
  for (float i = 0; i<100.0; i+=1.0) {
    if (int(i) % 16 == 0) { // por default el buffer puede almacenar hasta 64 bytes. Con el if se llama a flush para vaciar el buffer unicamente cuando este esta completamente lleno
      Serial.flush();
    }
  }
```

```

    sendFloat(i);
}
Serial.flush(); // se vacia todo lo que quede en el buffer de transmision
t1 = millis() - t0;
Serial.println( );
Serial.print( Tiempo de envio total: );
Serial.println(t1);
t1 = micros();
delayMicroseconds(20000 - t1);
}

```

5.4. Código de ensayo del servo y mapeo de ancho de pulso a ángulo

Se incluye una adaptación del programa `tope-subeybaja.ino`, que recibe por puerto serie un ancho de pulso entero entre 550 y 1600 μs (terminado en salto de línea), mantiene la lectura de la IMU a 100 Hz y muestra el ángulo estimado por el mapeo y el medido por el filtro. El ángulo medido corresponde al instante del registro y no implica que el movimiento haya terminado. Se debe usar la misma referencia horizontal que en la calibración de la tabla 3.1. La constante `PERIODO_CONTROL_US` permite comparar actualizaciones cada 20 ms con actualizaciones cada 1 s ; los pulsos siguen siendo generados por `Servo`. Si llegan varias consignas dentro de un período, se aplica la última. También se incluye la función inversa para obtener el ancho de pulso a partir de un ángulo de barra.

```

#include <Servo.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <math.h>

Servo miServo;
Adafruit_MPU6050 mpu;

const byte PIN_SERVO = 9;
// Usar 1000000UL para actualizar la consigna a 1 Hz.
// Servo mantiene su refresco nominal de 20 ms en ambos casos.
const unsigned long PERIODO_CONTROL_US = 20000UL;
unsigned long ultimoControlUs;
unsigned long ultimoRegistroUs;
int pulsoPendiente = 900;
int pulsoAplicado = 900;
bool descartarEntrada = false;

// Modelo calibrado del angulo de la BARRA, no del eje del servo.
float pulsoAAngulo(int pulsoUs) {
    pulsoUs = constrain(pulsoUs, 550, 1600);
    if (pulsoUs <= 900) return (pulsoUs - 900) * (14.8f / 350.0f);
    return (pulsoUs - 900) * (22.3f / 700.0f);
}

int anguloAPulso(float grados) {
    grados = constrain(grados, -14.8f, 22.3f);
    const float pulso = grados <= 0.0f
        ? 900.0f + grados * (350.0f / 14.8f)
        : 900.0f + grados * (700.0f / 22.3f);
    return (int)lroundf(pulso);
}

const unsigned long SERIAL_BAUD = 115200;
const unsigned long PERIODO_IMU_US = 10000; // 100 Hz
const float ALFA = 0.92f; // Peso del giroscopio

unsigned long ultimaLecturaUs;
float angulo = 0.0f;

```

```

// Buffer chico para recibir el ancho de pulso sin bloquear el muestreo IMU.
char entrada[12];
byte cantidadEntrada = 0;

float anguloAcelerometro(const sensors_event_t &accel) {
    // Inclinacion alrededor del eje X, en grados.
    return atan2(accel.acceleration.y, accel.acceleration.z) * 180.0f / PI;
}

void actualizarAngulo() {
    const unsigned long ahoraUs = micros();
    const unsigned long transcurridoUs = ahoraUs - ultimaLecturaUs;
    if (transcurridoUs < PERIODO_IMU_US) return;

    ultimaLecturaUs = ahoraUs;

    sensors_event_t accel, gyro, temperatura;
    mpu.getEvent(&accel, &gyro, &temperatura);

    const float dt = transcurridoUs * 1.0e-6f;
    // La libreria entrega rad/s; primero se convierte a grados/s.
    const float prediccionGyro = angulo + gyro.gyro.x * 180.0f / PI * dt;
    const float medicionAccel = anguloAcelerometro(accel);
    angulo = ALFA * prediccionGyro + (1.0f - ALFA) * medicionAccel;
}

void responderComando(const char *texto) {
    char *fin;
    const long anchoPulso = strtol(texto, &fin, 10);
    if (fin == texto || *fin != '\0' || anchoPulso < 550 || anchoPulso > 1600) {
        Serial.println( ERROR: envie un entero entre 550 y 1600 us );
        return;
    }
    pulsoPendiente = (int)anchoPulso;
}

void leerPuertoSerie() {
    // Presupuesto acotado para no bloquear indefinidamente la lectura IMU.
    for (byte n = 0; n < 16 && Serial.available() > 0; ++n) {
        const char caracter = Serial.read();
        if (caracter == '\n' || caracter == '\r') {
            if (descartarEntrada) {
                Serial.println( ERROR: linea demasiado larga );
            } else if (cantidadEntrada > 0) {
                entrada[cantidadEntrada] = '\0';
                responderComando(entrada);
            }
            cantidadEntrada = 0;
            descartarEntrada = false;
        } else if (!descartarEntrada) {
            if (cantidadEntrada < sizeof(entrada) - 1) {
                entrada[cantidadEntrada++] = caracter;
            } else {
                descartarEntrada = true;
            }
        }
    }
}

void actualizarControl() {
    const unsigned long ahoraUs = micros();
    if (ahoraUs - ultimoControlUs < PERIODO_CONTROL_US) return;
    ultimoControlUs = ahoraUs;
    pulsoAplicado = pulsoPendiente;
}

```

```

    miServo.writeMicroseconds(pulsoAplicado);
}

void registrar() {
    const unsigned long ahoraUs = micros();
    if (ahoraUs - ultimoRegistroUs < 100000UL) return;
    ultimoRegistroUs = ahoraUs;
    Serial.print( Pulso_us: );
    Serial.print(pulsoAplicado);
    Serial.print( , Modelo_deg: );
    Serial.print(pulsoAAngulo(pulsoAplicado), 3);
    Serial.print( , IMU_deg: );
    Serial.println(angulo, 3);
}

void setup() {
    miServo.attach(PIN_SERVO, 550, 1600);
    miServo.writeMicroseconds(900);

    Serial.begin(SERIAL_BAUD);
    Wire.begin();
    Wire.setClock(400000);

    pinMode(LED_BUILTIN, OUTPUT);
    if (!mpu.begin()) {
        Serial.println( ERROR: no se encontro la MPU6050 );
        digitalWrite(LED_BUILTIN, HIGH);
        while (true) delay(100);
    }

    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);

    // Arranca el filtro desde el angulo observado por el acelerometro.
    sensors_event_t accel, gyro, temperatura;
    mpu.getEvent(&accel, &gyro, &temperatura);
    angulo = anguloAcelerometro(accel);
    ultimaLecturaUs = micros();
    ultimoControlUs = ultimaLecturaUs;
    ultimoRegistroUs = ultimaLecturaUs;

    Serial.println( Envie un ancho de pulso en us y presione Enter (ej: 850) );
}

void loop() {
    actualizarAngulo();
    leerPuertoSerie();
    actualizarControl();
    registrar();
}

```